Multiplayer Tic-Tac-Toe — OCSF + EventBus

Software Engineering, Spring 2025–2026 — Lab 7

Waseem Sheety (205733322) • Adam (318407889)

Repository

https://github.com/SWeng-TutorialW/tic-tac-toe-waseemadam (code on the **main** branch; runnable JARs under `jars/`).

Short explanation of the implementation

We implemented a two-player networked Tic-Tac-Toe on the **OCSF** framework with a **JavaFX** client, keeping the course's three-module structure: **entities** (shared serializable classes and the protocol), **server** (OCSF `AbstractServer`), and **client** (OCSF `AbstractClient` + JavaFX). The OCSF framework classes themselves were not modified.

Server. `SimpleServer` extends `AbstractServer`. The first client to connect is told to wait; when a second connects, a `GameSession` randomly assigns X and O and randomly chooses who starts, then notifies both clients. The server is authoritative: it validates every move, updates the board, and broadcasts the move and whose turn is next to both players. It detects a win (and sends the winning line) or a draw, supports a rematch, refuses a third player, and if one player disconnects it returns the other to the waiting room.

Client. `SimpleClient` extends `AbstractClient` and implements `handleMessageFromServer`; it turns each server message into an **EventBus** event (the mediator / publisher–subscriber pattern). `GameController` subscribes to those events and updates the JavaFX board on the application thread. The connect screen lets the user type the server's host/IP and port, so the game can run across two machines.

How to run

Requires JDK 17 (or newer) and Maven. From the project root:

```
mvn clean install
```

Then, in separate terminals:

```
server →  cd server && mvn exec:java  (listens on port 3000)
player 1 →  cd client && mvn javafx:run
player 2 →  cd client && mvn javafx:run
```

In each client press **Connect** (host `127.0.0.1`, port `3000`). **Two computers:** run the server on one machine, find its IP (`ipconfig getifaddr en0` on macOS), then on the other machine run the client jar (`java -jar client-0.0.1-SNAPSHOT.jar`) and connect to that IP on port 3000.

Running on two different computers

Two separate computers connected over the local network - the server plus both clients, showing the same synchronized game.